

Системный анализ архитектуры, игровых механик и программного обеспечения цифровой адаптации книги-игры Подземелья Чёрного замка

Архитектурно-технический стек проекта и жизненный цикл веб-приложения

Цифровая адаптация интерактивного произведения Дмитрия Браславского «Подземелья Чёрного замка» (ремейк 2018 года классического издания 1991 года) представляет собой программный комплекс, функционирующий полностью на стороне клиента.¹

Архитектурное проектирование системы ориентировано на концепцию автономности и независимости от серверной инфраструктуры (offline-first).¹ Это исключает сетевые задержки при переходах между игровыми сценами и гарантирует полную работоспособность приложения в условиях отсутствия связи.¹

Сборка игрового приложения осуществляется с помощью автоматизированного bash-сценария build.sh, выполняющего слияние исходных компонентов из рабочей директории src/ в единый исполняемый дистрибутив podzemelye-chyornogo-zamka-remake.html размером около 9,3 МБ.¹ В процессе компиляции текстовые данные, логика игрового процесса, стилистическое оформление, наборы шрифтов (Cinzel, Cormorant, Veles Redone, Cyrillic Old Face) и графические активы в кодировке Base64 объединяются в монолитную структуру.¹ Техническое решение включает поддержку прогрессивных веб-приложений (PWA), декларацию метаданных в манифесте веб-приложения, Service Worker для локального кэширования ресурсов и генерацию процедурных звуковых эффектов посредством встроенного Web Audio API.¹ Для старых или ограниченных сред выполнения предусмотрена интеграция сжатых аудиофайлов в формате OGG.¹

Распределение компонентов в репозитории проекта и их функциональная роль в процессе развертывания представлены в следующей таблице:

Директория репозитория	Типы файлов и форматы данных	Роль в жизненном цикле приложения	Ссылка на источники
dist/	Монолитный HTML-файл приложения,	Продакшн-версия системы, готовая к запуску на	¹

	Service Worker (sw.js), манифест PWA (manifest.webmanifest), процедурный аудио-пакет OGG	стороне клиента в автономном режиме	
src/	Логика приложения (game_logic.js), интерфейс создания персонажа (game_shell_top.html), картографический модуль (map_module.js), Base64-графика	Среда активной разработки; содержит модули, подлежащие сборке и оптимизации	1
assets/	Файл электронной книги FB2, исторические сканы книги, размеченный текст (book_text.md), файл логических коррекций (text_corrections.json)	Архивные и справочные материалы, содержащие исходные тексты и списки логических правок	1
art-pack/	Скрипты на языке Python для обработки метаданных изображений, CDN-адреса, сопоставления иллюстраций	Инструменты управления графическими активами и генерации изображений через Midjourney	1

docs/	Спецификации генерации изображений, результаты графового аудита игровых сцен, дорожная карта развертывания PWA	Техническая документация, описывающая топологию графа переходов и стратегии деплоя	1
_handoff/	Брифы и проектные задачи для сохранения контекста между сессиями разработки	Координация и сохранение проектного контекста между агентами разработки	1

Математическое моделирование игровых состояний и теория вероятностей в механиках

Детерминированное моделирование игрового процесса базируется на математических функциях генерации характеристик персонажа и вероятностном расчете боевых столкновений.¹ Начальное состояние игрока формируется на этапе создания персонажа посредством дискретного равномерного распределения результатов бросков

шестигранных игровых костей ($d6$).¹

Пусть X_i — независимая случайная величина, обозначающая результат броска одной игровой кости, где $X_i \in \{1, 2, 3, 4, 5, 6\}$ и вероятность получения любого значения составляет $P(X_i = k) = \frac{1}{6}$ для каждого k . Исходные параметры героя рассчитываются по следующим формулам:

- Мастерство (S_{skill}): $S_{\text{skill}} = X_1 + 6$.¹ Диапазон значений составляет \$\$, а математическое ожидание $E = 3.5 + 6 = 9.5$.
- Выносливость (S_{tamina}): $S_{\text{tamina}} = X_2 + X_3 + 12$.¹ Диапазон значений равен \$\$, математическое ожидание составляет $E = 3.5 + 3.5 + 12 = 19.0$.
- Удача (L_{uck}): $L_{\text{uck}} = X_4 + 6$.¹ Диапазон значений составляет \$\$, математическое ожидание равно $E[L_{\text{uck}}] = 3.5 + 6 = 9.5$.

Использование стандартной функции JavaScript Math.random() для генерации этих параметров несет в себе риски предсказуемости результатов из-за псевдослучайного характера алгоритмов на основе периодических сдвигов.¹ Чтобы гарантировать криптографическую стойкость и предотвратить возможность манипулирования вероятностями со стороны пользователя, программный движок использует метод crypto.getRandomValues() в рамках операции взятия остатка от деления.¹ Это обеспечивает высокую энтропию и математически точное равномерное распределение вероятностей при каждом броске.¹

Боевые столкновения рассчитываются пошагово.¹ В каждом раунде вычисляется Сила

удара игрока (C_{player}) и противника (C_{enemy}):¹

$$C_{\text{player}} = Y_1 + Y_2 + S_{\text{kill_current}}$$

$$C_{\text{enemy}} = Y_3 + Y_4 + S_{\text{kill_enemy}}$$

где $Y_j \sim d6$ — случайные величины броска костей для боя.¹ Противоборствующая сторона с меньшим значением Силы удара теряет 2 единицы Выносливости.¹ Принятие игроком решения о бегстве из боя наказывается автоматическим вычитанием

Выносливости: $S_{\text{tamina}} \leftarrow S_{\text{tamina}} - 2$.¹

Проверка Удачи представляет собой случайный эксперимент, успешность которого

зависит от текущего значения характеристики L_c .¹ Результат броска двух костей

$Z = Y_1 + Y_2$ сравнивается с текущим лимитом: испытание считается успешным, если

$Z \leq L_c$.¹ Сразу после завершения проверки значение Удачи уменьшается на единицу:

$L_{t+1} = L_t - 1$.¹ Данная механика описывает марковский процесс с декрементом параметра, где вероятность успешного прохождения последующих проверок нелинейно снижается, стимулируя игрока к экономному расходованию этого ресурса на ранних этапах прохождения.¹

Вероятностные характеристики игровых костей и ключевых параметров представлены в таблице:

Игровое событие	Математическая формула события	Интервал возможных исходов	Математическое ожидание исхода	Вероятность успеха при среднем значении	Источник

				характеристики	
Генерация Мастерства	$S_{kill} = d6 +$	\$\$	9.5	—	1
Генерация Выносливости	$S_{tamina} = d$	\$\$	19.0	—	1
Генерация Удачи	$L_{uck} = d6 -$	\$\$	9.5	—	1
Базовая Сила удара	$Combat_D$	\$\$	7.0	—	1
Проверка Удачи (исходная)	$P(2d6 \leq$	$2d6 \leq$	—	$\approx$ (30 из 36 комбинаций)	1
Проверка Удачи (после 3 тестов)	$P(2d6 \leq L$	$2d6 \leq$	—	$\approx$ (15 из 36 комбинаций)	1

Функциональная спецификация программного движка и управление игровым состоянием

Архитектура программного движка, реализованная в файле game_logic.js, построена вокруг централизованного управления состоянием приложения через глобальный изменяемый объект S.¹ Для обеспечения безопасности и предотвращения несанкционированного доступа со стороны внешних скриптов, переменная S изолирована внутри замыкания, а взаимодействие с ней внешних модулей контролируется посредством геттера и сеттера, определенных в глобальном объекте window через Object.defineProperty.¹ Это гарантирует стабильную связь с визуальным модулем карты без риска повреждения структуры данных.¹

Инициализация новой игровой сессии через фабрику состояний initState формирует

строغو типизированную структуру данных.¹ Емкость рюкзака изначально ограничена семью слотами (bagSize: 7), однако в коде предусмотрены условия для динамического расширения инвентаря до девяти ячеек при приобретении улучшенного походного снаряжения.¹ Поле pending_combat_buff служит временным буфером для накопления боевых модификаторов характеристик персонажа, активируемых при чтении определенных параграфов или использовании магии перед началом раунда.¹

Для сохранения игрового прогресса применяется сериализация текущего состояния S в текстовый формат JSON с последующей записью в хранилище браузера localStorage под ключом podzch_v5.¹ Для поддержки обратной совместимости при загрузке сохраненных файлов движком вызывается процедура автоматической миграции версий.¹ В случае обнаружения структуры данных четвертой версии движок выполняет конвертацию: значение luck дублируется в новое обязательное поле максимального лимита luckMax, а устаревшие или неиспользуемые переменные, такие как массивы логических переключателей luckBoxes, безвозвратно удаляются из памяти.¹

Важным элементом обеспечения отказоустойчивости движка при импорте внешних файлов сохранений является функция валидации normalizeSave(s).¹ Данная процедура выполняет глубокую проверку структуры импортированного объекта, принудительно восстанавливая целостность массивов инвентаря, выбранных заклинаний, логов событий и истории посещенных параграфов, если они были повреждены или удалены при ручном редактировании файла пользователем.¹ Это исключает возникновение фатальных ошибок времени выполнения (runtime exceptions) при обращении к неопределенным переменным (undefined).¹

Магическая система Мэйлина, доступная игроку, оперирует пулом из восьми заклинаний, выбираемых на этапе генерации персонажа (максимум 10 слотов).¹ Каждое заклинание представляет собой условный триггер, изменяющий стандартные алгоритмы обработки игрового состояния¹:

- LEVITATION (Левитация): Позволяет полностью игнорировать ловушки провала и падения в пропасть, отключая вызов функций нанесения урона на целевых параграфах.¹
- FIRE (Огонь): Служит средством дистанционного поражения, позволяя наносить урон противникам до начала стандартной фазы боя.¹
- ILLUSION (Иллюзия): Изменяет логику диалогов и позволяет мирно обходить опасных персонажей.¹
- FORCE (Сила) и WEAKNESS (Слабость): Воздействуют непосредственно на переменные СИЛЫ УДАРА, записывая положительные модификаторы в буфер pending_combat_buff или ослабляя параметры соперника.¹
- COPY (Копия): Создает дубликат врага, перенаправляя боевой урон на клона.¹
- HEALING (Лечение): Восстанавливает 8 единиц Выносливости, однако логика движка содержит жесткое условное ограничение, блокирующее вызов этого метода, если флаг активности боя равен значению «истина».¹
- SWIMMING (Плавание): Дает возможность преодолевать водные преграды; движок

отслеживает перемещения героя и автоматически деактивирует это состояние, как только номер текущей секции указывает на возвращение персонажа на сушу.¹

Логико-пространственная модель игрового мира и синхронизация слоев

Логическая структура игрового мира, описанная в модуле `map_module.js`, представляет собой ориентированный граф, узлы которого соответствуют ключевым географическим точкам на карте.¹ Весь игровой мир разделен на три изолированных пространственных слоя, каждый из которых имеет собственную систему координат для визуального отображения на карте¹:

- Слой `overworld` (Внешний мир / Зачарованный лес): Координатная сетка размером 1600 на 1000 пикселей.¹
- Слой `castle_exterior` (Подходы к Чёрному замку): Координатная сетка размером 1200 на 900 пикселей.¹
- Слой `castle_interior` (Внутренние покои замка): Координатная сетка размером 1600 на 1200 пикселей.¹

Каждый узел графа (node) описывается набором свойств, включающим уникальный строковый идентификатор, координатную привязку к пиксельной сетке текущего слоя, тип местности (перекресток, развилка, здание, река, комната), массив текстовых тегов для категоризации событий и массив ссылок на параграфы книги `paragraph_refs`.¹ Привязка пространственной карты к логике книги-игры осуществляется через сквозное отслеживание изменений переменной состояния `S.section`.¹

Когда игрок переходит на новый параграф, картографический модуль сканирует массив узлов на предмет совпадения активной секции с массивом `paragraph_refs`.¹ При обнаружении совпадения статус узла в структуре карты меняется на «посещенный», иницилируя процесс динамического удаления эффекта «тумана войны». ¹ Логика раскрытия карты описывается правилом `reveal_policy: "show_stub_from_discovered"`.¹ Это означает, что при активации узла система отрисовывает не только саму точку, но и частично приоткрывает связанные с ней ребра графа (edges), давая игроку визуальное представление о возможных направлениях дальнейшего движения, в то время как целевые узлы на концах этих ребер остаются скрытыми до момента непосредственного перевода игровой сессии на соответствующий параграф.¹

Систематизация логических коллизий и реестр корректировок игрового баланса

Для приведения оригинального текста книги-игры к строгим математическим правилам цифрового движка используется файл корректировок `text_corrections.json`.¹ Он выступает в качестве связующего звена между текстовым наполнением `book_text.md` и исполняемой логикой `game_logic.js`, преобразуя абстрактные описания событий в конкретные изменения игрового состояния.¹

Подробный анализ примененных корректировок (группы 1–18) и механизмов их

интеграции с программным движком представлен в таблице:

Идентификатор группы / Пункта	Суть и нарративный контекст логической коллизии	Алгоритмический метод обработки в game_logic.js	Системное влияние на игровой баланс и прохождение	Ссылка на источники
Группа 1.3	Недостающие автоматические эффекты в параграфах с ловушками	Преобразование текстовых ранений в прямое вычитание Выносливости (например, stamina: -5)	Исключение ошибок, при которых физический урон приводил к некорректной потере предметов	¹
Группа 1.4	Использование веревки без ее фактического наличия (§412, §464)	Добавление условной проверки наличия предмета в инвентаре перед отображением вариантов выбора	Устранение логических дыр, позволявших обходить препятствия без необходимого снаряжения	¹
Группа 3	Альтернативные способы получения золотого ключа	Добавление свойства автоматического получения предмета в параграфах §140, §440, §1172	Обеспечение возможности прохождения сюжета через альтернативные ветки	¹
Группа 4	Неверная адресация графических	Жесткое сопоставление номеров	Синхронизация визуального ряда с	¹

	ресурсов	параграфов (например, §1072) с каноническим и идентификаторами изображений	текущим описанием игрового окружения	
Группа 6.1 (fish_help)	Логика взаимодействия с Золотой Рыбкой	Установка флага помощи при выборе соответствующей ветки и проверка флага на заблокированных участках	Предотвращение прохождения квестов без предварительного спасения персонажа	1
Группа 6.3 (candle_lamp)	Преодоление темноты в коридорах замка	Начисление предмета «Свеча» при нахождении и условная проверка его наличия в темных зонах	Предотвращение тупиковых ситуаций в подземельях после прохождения лагеря гоблинов	1
Группа 6.5 (castle_key)	Контроль доступа к заблокированным дверям	Начисление ключа Чёрного замка и блокировка связанных переходов при его отсутствии	Гарантия детерминированного исследования внутренних помещений крепости	1
Группа 6.6 (bear_key)	Разрыв связей переходов из-за ошибки перевода	Коррекция текстового наименования на «Медный	Устранение критического тупика, отсекавшего	1

	ключа	ключик» и восстановлен ие более 40 сломанных ссылок	значительную часть игрового контента	
Группа 6.8 (thread_ball)	Сбой арифметическ ого перехода при использовани и клубочка	Коррекция знака математическ ого смещения в тексте: замена модификатора с −50 на +50	Восстановлен ие работоспособ ности квестовой цепочки с использовани ем нити	¹
Группа 11	Невозможнос ть приобретения серебряного браслета	Интеграция покупки предмета в ассортимент товаров крестьянина в параграфе §340	Восстановлен ие доступа к защитному артефакту, который ранее нельзя было купить	¹
Группа 12	Использовани е бронзового свистка	Приведение различных текстовых названий предмета к единой строке и блокировка выборов	Исключение сюжетной путаницы при активации звукового артефакта	¹
Группа 13	Ранение от ловушки с метательными ножами	Изменение типа события с «потери ножей из инвентаря» на	Корректное отражение физического урона вместо ошибочного	¹

	(§1122)	«нанесение —5 урона Выносливости »	изъятия экипировки	
Группа 14	Отсутствие интерактивно о интерфейса торговли	Внедрение схемы {purchase: true, cost: X, acquires: Y} для обработки транзакций	Автоматизаци я покупок и балансировка золотого запаса игрока во всех лавках	¹
Группа 15	Автоматическ ое получение трофеев после боя	Расширение структуры переходов свойством автоматическ ого начисления предметов победителя	Избавление игрока от необходимост и вручную добавлять трофеи после победы в дуэлях	¹
Группа 16	Проверка финансовой состоятельно сти	Блокировка отображения кнопок покупки при недостаточно м количестве золота на балансе	Предотвраще ние возможности приобретения квестовых предметов при отсутствии средств	¹
Группа 17	Функционал фигурного ключа	Добавление ключа в магазин (§340) и условная валидация	Корректное связывание квестового ключа с механизмами дверей	¹

		замочных скважин	Чёрного замка	
Группа 18	Открытие люка в параграфе §774	Гейтинг перехода условием проверки наличия в рюкзаке красивого куска дерева	Восстановление логики использования найденных деревянных брусков	¹

Системные выводы и стратегический план верификации

Текущее состояние разработки цифровой адаптации книги-игры «Подземелья Чёрного замка» характеризуется высоким уровнем готовности ядра системы и успешным решением большинства логических проблем первоисточника.¹ Программный движок эффективно справляется с управлением состоянием и обеспечивает корректную интеграцию игровых механик, структуры текста и картографического модуля.¹

Для успешного вывода проекта в финальный продакшн-релиз необходимо реализовать комплекс мероприятий, направленных на повышение стабильности и расширение функционала системы¹:

- **Активация прогрессивного веб-приложения (PWA):** Требуется размещения собранного дистрибутива на веб-ресурсе с поддержкой защищенного протокола HTTPS.¹ Без выполнения этого условия современные браузеры блокируют регистрацию Service Worker, что делает невозможным установку приложения на мобильные устройства и его работу в автономном режиме.¹
- **Автоматизация сквозного тестирования:** Рекомендуется разработать автоматический парсер игрового графа (на базе алгоритмов поиска в ширину BFS или Дейкстры), способный имитировать действия игрока и проходить все возможные маршруты от параграфа §1 до финального параграфа §1220.¹ Это позволит гарантировать достижимость всех 60 концовок, подтвердить отсутствие изолированных параграфов и верифицировать корректность работы условных проверок инвентаря на критических сюжетных развилках.¹
- **Оптимизация аудиосистемы:** Процедурный синтезатор звука на базе Web Audio API решает задачу минимизации размера файла, однако качество генерируемых эффектов уступает студийным записям.¹ Необходимо провести работу по замене процедурного звукового пакета на оптимизированные аудиофайлы формата OGG под свободными лицензиями (CC0/CC-BY), сохранив при этом высокую скорость загрузки приложения.¹

- **Масштабируемость хранилища данных:** Текущий механизм сохранения прогресса использует `localStorage`, емкость которого в мобильных браузерах ограничена 5 МБ.¹ Для поддержки ведения расширенных логов событий и хранения истории нескольких игровых персонажей целесообразно перевести систему на использование хранилища `IndexedDB`, обеспечивающего больший объем квоты дискового пространства и асинхронный доступ к данным.¹
- **Мультиязычная локализация:** Реализация заявленной поддержки английского и французского языков потребует выноса всех строковых литералов, названий предметов и текстов параграфов из файлов `game_logic.js` и `book_text.md` во внешние словари локализации (`i18n`-файлы конфигурации `JSON`).¹ Это позволит динамически переключать язык интерфейса без необходимости создания дублирующих сборок приложения.¹

Источники

1. `README.md`